

Universidade de Caxias do Sul · Engenharia de Software

Disciplina: Integracao de Aplicacoes · Trabalho Final

Sistema de Mensageria com Apache Kafka

Simulacao do Fluxo de Skip de Introducao da Netflix

Professora: **Marines Refosco Severo Bertola** | Aluno: **Mauricio Knabben** | Junho 2026

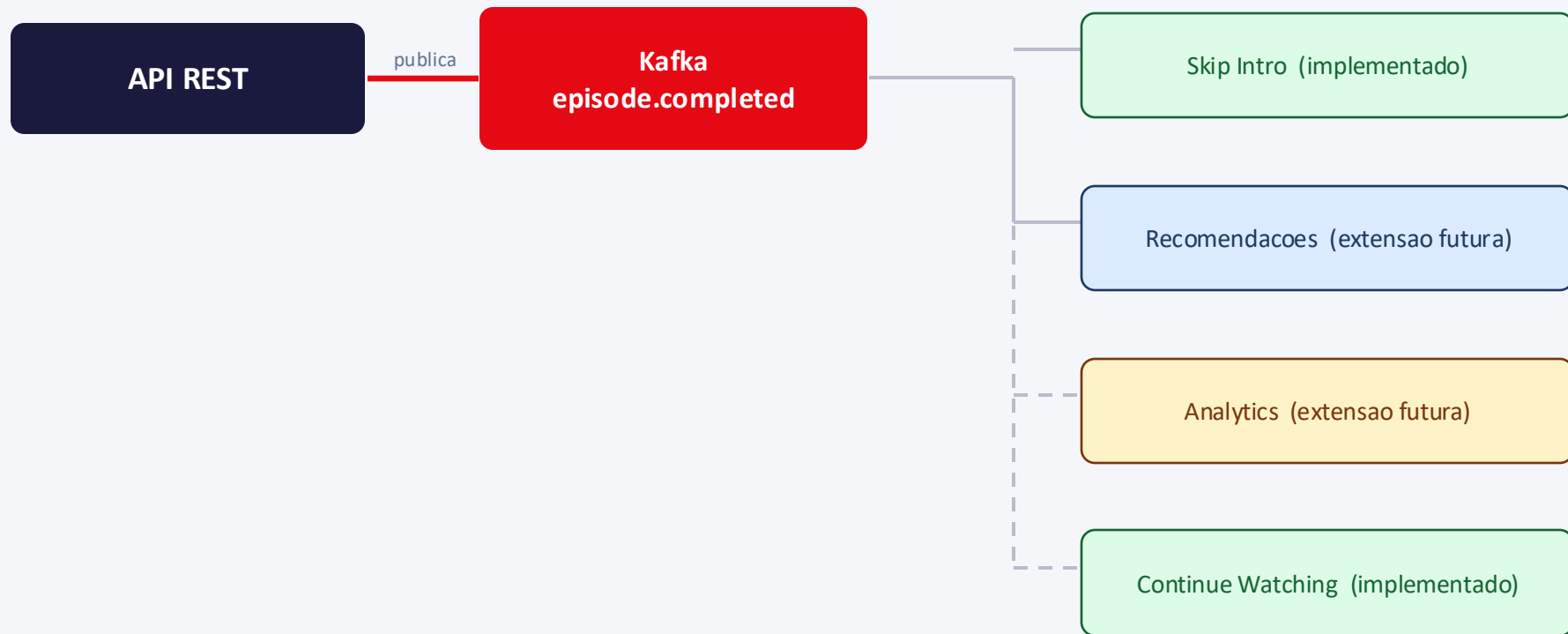
0 Problema: O Duplo Clique

- 1 Usuario clica em "Proximo Episodio"
- 2 Player carrega o proximo episodio a partir do segundo 0
- 3 Introducao da serie começa do inicio novamente
- 4 **Usuario precisa clicar em "Pular Introducao" outra vez**

Atrito desnecessario: o usuario ja demonstrou intencao de continuar assistindo

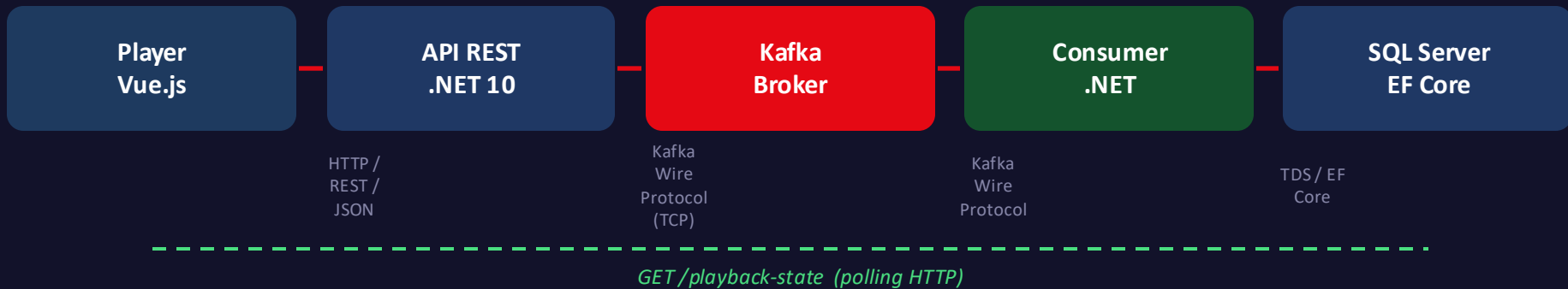


Por que Kafka? Desacoplamento entre Sistemas



Adicionar um novo consumer nao exige alterar uma linha do produtor — principio central em plataformas como a Netflix

Arquitetura da Integracao



Eventos de dominio:

- EpisodeCompletedEvent
- PlaybackStartedEvent
- PlaybackPositionEvent (heartbeat a cada 30s)

Padroes aplicados:

- At-least-once delivery
- Idempotencia (unique index)
- CQRS simplificado
- Session anchor (SessionId)

Fluxo de Integracao: Camadas de Protocolo

CAMADA DE APRESENTACAO

HTTP / REST + JSON
(porta 80 / 5000)

Vue.js 3
Player

POST /completed

GET /playback-state
(polling)

ASP.NET Core
API REST

CAMADA DE MENSAGERIA

Kafka Wire Protocol
(TCP 9092 via librdkafka)

Episode
Producer

Apache Kafka
episode.completed

Consumer
skip-intro-group

CAMADA DE DADOS

EF Core ORM / TDS
(localhost,1433 SQL Server)

Consumer
ProcessEvent()

EF Core
DbContext (Scoped)

SQL Server
PlaybackStates

ZooKeeper / ZAB (Zookeeper Atomic Broadcast): coordenacao do cluster Kafka — eleicao de lider, deteccao de falhas, sincronizacao de replicas

Tecnologias Utilizadas

Apache Kafka

Broker — Publish-Subscribe

Confluent.Kafka

Cliente .NET / Kafka Wire
Protocol (TCP)

.NET 10 / ASP.NET Core

API REST + BackgroundService

Entity Framework Core

ORM — transformacao
objeto/relacional (TDS)

SQL Server

Persistencia de PlaybackState e
WatchProgress

Vue.js 3

Player web + console de
mensageria

Docker Compose

Kafka + ZooKeeper + SQL Server
em containers

ZooKeeper

Coordenacao do cluster Kafka
via ZAB

Implementacao: Destaques Tecnicos

At-Least-Once Delivery

`EnableAutoCommit = false`

Commit manual so apos processar o evento. Em caso de falha, a mensagem e reprocessada automaticamente.

Idempotencia no Banco

Indice unico (UserId, EpisodeId)

Duplicatas viram UPDATE em vez de INSERT — o estado final sempre consistente.

Session Anchor

SessionId = UUID por sessao de browser

Evita que um skip pre-computado em sessoes anteriores seja aplicado numa nova abertura.

Pipeline de transformacao de dados:

Record C# --> JSON (System.Text.Json) --> Kafka Wire Protocol (binario) --> Record C# --> EF Core Entity --> SQL (TDS)

Codigo Real: Consumer e Produtor Kafka

Consumer — EpisodeConsumer.cs

```
// Consumer Group + At-Least-Once
GroupId      = "skip-intro-group",
EnableAutoCommit = false; // at-least-once
delivery
```

```
// IServiceScopeFactory — DbContext em Singleton
using var scope = _scopeFactory.CreateScope();
var repo = scope.ServiceProvider
    .GetRequiredService<IPlaybackStateRepo>();
repo.SaveAsync(state).GetAwaiter().GetResult();
```

```
// Commit manual: so apos processar
_consumer.Commit(msg);
```

Produtor — POST /episodes/{id}/completed

```
// Constroi e publica o evento de dominio
var evento = new EpisodeCompletedEvent(
    UserId:      userId,
    NextEpisodeId: episodeId + 1,
    SessionId:    sessionId);
await _producer.PublishAsync(evento);
return Accepted(new { userId, sessionId });
```

```
// Evento = contrato entre Produtor e Consumer
public record EpisodeCompletedEvent(
    Guid      UserId, int EpisodeId,
    int       NextEpisodeId,
    Guid      SessionId,
    DateTime CompletedAt);
```

O evento e o unico contrato entre Produtor e Consumer.
A API nao conhece o Consumer — desacoplamento total.
Adicionar um novo consumer nao altera o produtor.

Demonstracao e Resultados

Fluxo testado via arquivo .http ou player web:

- 1 GET /catalog/episodes
- 2 POST /episodes/1/completed (publica no Kafka)
- 3 (~1s) Consumer salva PlaybackState: startAt = 70s
- 4 GET /episodes/2/playback-state --> source: "pre-computed"
- 5 **Player inicia EP2 no segundo 70 -- intro pulada**

Consumers / skip-intro-group

State	Members	Assigned Topics	Assigned Partitions	Coordinator ID	Total lag
STABLE	1	1	1	1	0

Q Search by Topic Name

Topic	Consumer Lag
episode.completed	0

► Console de Mensageria video.readyState: 0 (HAVE_NOTHING) | currentTime: 124.5s

```
19:28:10.367 INFO App montada - carregando EP1
19:28:10.368 DEBUG sessionId: 919b64d5-e5c7-465e-abdf-d9229d60039b
19:28:10.368 DEBUG userId: 5d0f3950-6906-45ac-a54f-699754e90db1
19:28:10.368 INFO -- loadEpisode(id=1, startAt=0s) --
19:28:10.369 INFO GET /catalog/episodes/1
19:28:10.409 DEBUG loadedmetadata - duration: 169.8s, readyState: 4
19:28:10.509 INFO Episódio: "Piloto" | intro: 0s-70s
19:28:10.509 INFO Chamando play()... readyState=4
19:28:10.511 SUCCESS play() OK - reprodução iniciada
19:28:10.545 INFO PlaybackStartedEvent publicado -> EP1, startAt=0s, sessionId=919b64d5...
```

Conclusao

01

Problema real resolvido

O duplo clique para pular intro foi eliminado via pre-computacao assincrona — sem bloquear o usuario.

02

Desacoplamento genuino

O mesmo EpisodeCompletedEvent alimenta multiplos consumers independentes sem alterar o produtor.

03

Protocolos heterogeneos coexistindo

HTTP/REST, Kafka Wire Protocol e TDS — cada um escolhido pela natureza da integracao em cada camada.

04

Padrao que escala

A arquitetura implementada e a mesma base usada por centenas de microservicos em plataformas como a Netflix.